

Jagiellonian University
Department of Theoretical Computer Science

Oskar Krygier

**Implementation and comparison of
minimum cost flow algorithms**

Bachelor Thesis

Supervisor: dr hab. Krzysztof Turowski

June 2026

Contents

1	Introduction	3
1.1	Introduction	3
1.1.1	Protein Pocket Matching	3
1.1.2	An overview of the most notable results	4
1.2	Notations and assumptions	5
1.3	Reduced costs	7
1.4	Optimality conditions	8
1.4.1	Complementary slackness optimality conditions	9
1.5	Dual problem	9
1.6	Equivalent problems	11
1.6.1	Minimum cost circulation with lower bounds	11
1.6.2	Minimum cost circulation without lower bounds	12
1.6.3	Uncapacitated minimum cost flow	13
2	Edmonds-Karp algorithm	15
2.1	Successive shortest path algorithm	16
2.1.1	Correctness	17
2.1.2	Complexity	17
2.2	Capacity scaling	17
2.2.1	Correctness	19
2.2.2	Complexity	20
3	Orlin algorithm	21
3.1	Idea	21
3.1.1	Contraction	22
3.1.2	A difficult case for the algorithm with contraction	23
3.1.3	Optimized shortest path algorithm	25
3.2	Pseudocode	25
3.3	Correctness and complexity of the algorithm	26
3.3.1	Correctness	26
3.3.2	Complexity	27

4	Goldberg-Tarjan algorithm	29
4.1	Definitions	29
4.2	The algorithm	30
4.2.1	The successive approximation framework	30
4.3	Push-relabel	30
4.3.1	Complexity	32
4.4	The wave method	33
4.4.1	Complexity	34
5	Benchmarks	35
5.1	Running times and graphs	35
5.1.1	8-density	36
5.1.2	$\sqrt{n}$ -density	36
5.1.3	Increasing arc number	37
5.1.4	Increasing capacities	37
5.1.5	Increasing costs	38
5.1.6	Increasing source and target nodes	38
5.1.7	TCT benchmark	39
5.2	Summary	40

Chapter 1

Introduction

1.1 Introduction

The minimum cost flow problem is one of the most fundamental problems of network flows. In this problem we consider a capacitated network flow with a cost per unit assigned to every arc. Then, given a supply or demand for each node, the point is to find a flow of minimum cost.

One can observe that the shortest path problem and the maximum flow problem can be easily reduced to the minimum cost.

The problem found many applications in various fields, including transport [1], medicine [2], military [3], and many more. One such application is provided below to illustrate the issue.

1.1.1 Protein Pocket Matching

Protein structures possess surface cavities or depressions, known as *pockets*, which determine their capacity to bind with other molecules. A key objective is to evaluate how well computational models, such as AlphaFold [4], preserve these binding pockets compared to experimentally determined structures.

Assuming we have two aligned protein structures—one experimental and one predicted—we identify their pockets using a detection tool such as P2Rank [5] and extract the geometric midpoint of each pocket. Let A and B denote the sets of pocket midpoints for the experimental and predicted structures, respectively. For any pair of points $i \in A$ and $j \in B$, let d_{ij} represent their spatial distance.

Our goal is to match the closest corresponding pockets between the two structures. While traditional bipartite matching can be solved using maximum flow algorithms, our objective minimizes distance, requiring a minimum-cost flow formulation.

We construct a directed network containing a node for each midpoint in sets A and B . We insert directed arcs from every $i \in A$ to every $j \in B$, assigning each arc a cost of d_{ij} and infinite capacity. Next, we introduce a source node s

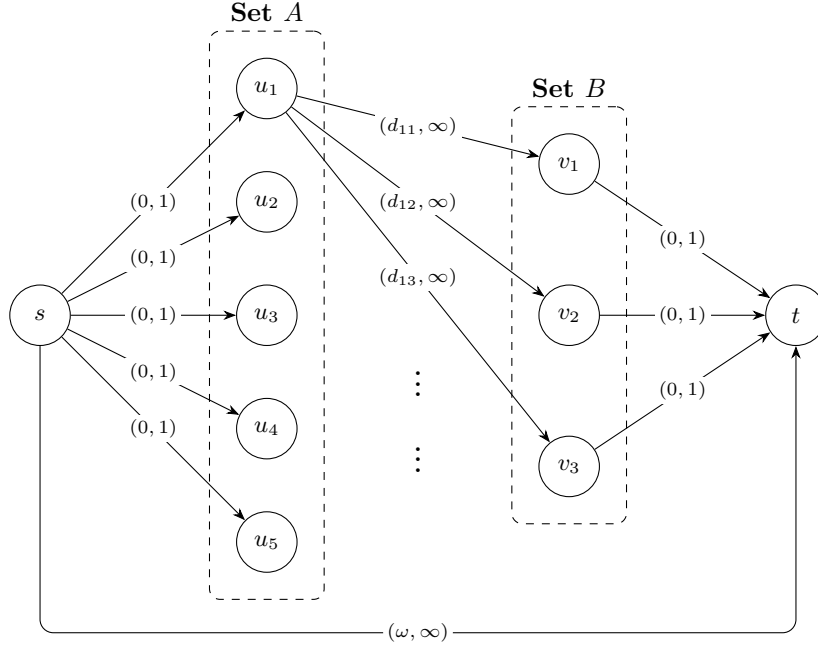


Figure 1: Protein structure pockets matching network

and a sink node t . We connect s to each node $i \in A$ using arcs with a cost of 0 and a capacity of 1. Similarly, we connect each node $j \in B$ to t with zero-cost, unit-capacity arcs.

Because the detection algorithm may identify pockets in one structure that are absent in the other, we must account for unmatched pockets. We address this by adding a bypass arc $s \rightarrow t$ with infinite capacity and a fixed penalty cost ω . Finally, letting $b(i)$ denote the supply or demand at node i , we set the source supply to $b(s) = \max\{|A|, |B|\}$ and the sink demand to $b(t) = -\max\{|A|, |B|\}$. Figure 1 illustrates the complete network architecture.

1.1.2 An overview of the most notable results

The minimum cost flow problem has been studied extensively, especially in the early 1990s. In recent years, researchers have developed many techniques to design more effective algorithms to solve the problem. In this section, we list some known algorithms and their running times.

There are many different approaches to the problem that could be divided into three distinct sections. The first, classical algorithms had *pseudo-polynomial* deterministic time complexity, then there were developed more sophisticated

polynomial deterministic time algorithms, and the latest, *randomized* polynomial time algorithms.

Let $U = \max_{e \in E} \{u_e\}$ denote the maximum capacity and let $C = \max_{e \in E} \{c_e\}$ denote the maximum arc cost. We call the algorithm *polynomial* when its running time is polynomially bounded in terms of n , m , $\log U$ and $\log C$ as the input network can be encoded on the polynomial number bits with respect to these quantities. We say that the algorithm is *strongly polynomial* if its time complexity is polynomially bounded only in terms of n or m .

Many algorithms reduce the minimum cost flow problems to simpler problems, including the *shortest path problem* (marked as *SP*), the *maximum flow problem* (*MF*) and the *negative cost cycle* (*NC*). When the time complexity of an algorithm is stated as $O(mn^2C) \cdot SP$, it means that the running time of the algorithm is dominated by solving the shortest path problem $O(mn^2C)$ times.

1.1.2.1 Classical pseudo-polynomial algorithms for MCF

In Table 1 we recall some classical minimum cost flow algorithms, running in pseudo-polynomial times.

Year	Running time	Reference
1960	$O(mn^2C) \cdot SP$	Iri [6], Busacker and Gowen [7]
1960	$O(mU) \cdot SP$	Minty [8], Fulkerson [9]
1962	$O(n \min\{U, C\}) \cdot (SP + MF)$	Ford and Fulkerson [10]
1967	$O(mC) \cdot NC$	Klein [11]
1969	$O(nU) \cdot SP$	Edmonds and Karp [12]

Table 1: Classical pseudo-polynomial algorithms for MCF from [13]

1.1.2.2 Polynomial algorithms

In the 1970s and 1980s several polynomial-time algorithms were developed for minimum cost flow. Their time complexity and authors are shown in Table 2. The algorithms in bold are the main topic of this work.

1.1.2.3 Randomized algorithms

In Table 3, we cite some recent randomized algorithms for minimum cost flow problem. Despite the lack of detailed description in this work, they are worth mentioning because they are current state-of-the-art MCF algorithms.

1.2 Notations and assumptions

Definition 1.1 (Graph and network [30]). A **directed graph** $G = (V, E)$ consists of a set V of nodes and a set E of arc whose elements are ordered pairs

Year	Running time	Reference
1972	$O(m \log U) \cdot SP$	Edmonds and Karp [14]
1980	$O(m \log C) \cdot MF$	Röck [15]
1984	$O(m^2 \log n) \cdot SP$	Orlin [16], Fujishige [17]
1985	$O(m^2 \log n) \cdot MF$	Tardos [18]
1986	$O(n^2 \log n) \cdot SP$	Galil and Tardos [19]
1987	$O(mn \log n \log U \log(nC))$	Gabow and Tarjan [20]
1987	$O(n^3 \log(nC))$	Goldberg and Tarjan [21]
1988	$O(m \log n) \cdot SP$	Orlin [22]
1988	$O(mn \log \log U \log(nC))$	Ahuja et al. [23]

Table 2: Polynomial time algorithms for MCF from [13]

Year	Running time	Reference
2008	$O(m^{3/2} \text{polylog } n \log U)$	Daitch and Spielman [24]
2014	$O(m\sqrt{n} \text{polylog } n \log^2 U)$	Lee and Sidford [25]
2021	$O((m \log(UC) + n^{1.5} \log^2(UC)) \text{polylog } n)$	van den Brand et al. [26]
2021	$O(m^{3/2-1/762} \text{polylog } n \log(U+C))$	Axiotis et al. [27]
2022	$O(m^{3/2-1/58} \text{polylog } n \log(U+C))$	van den Brand et al. [28]
2022	$O(m^{1+o(1)} \log U \log C)$	Chen et al. [29]

Table 3: Recent randomized algorithms for MCF from [13]

of distinct nodes. A **directed network** is a directed graph whose nodes and/or arcs have associated numerical values.

In this paper, we use terms *graph* and *network* interchangeably.

Definition 1.2 (Flow [30]). A **flow** x is a function $x: E \rightarrow \mathbb{R}$ for a given function $b: V \rightarrow \mathbb{R}$ that satisfies the following constraints:

- $\sum_{\{j: (i,j) \in E\}} x_{ij} - \sum_{\{j: (j,i) \in E\}} x_{ji} = b(i)$ for all $i \in V$,
- $0 \leq x_{ij} \leq u_{ij}$ for all $(i,j) \in E$.

Definition 1.3 (Minimum cost flow [30]). Let $G = (V, E)$ be a directed network with a cost c_{ij} and a capacity u_{ij} associated with every arc $(i,j) \in E$. For each node $i \in V$, we have a number $b(i)$, $b: V \rightarrow \mathbb{R}$ that indicates its **supply** ($b(i) > 0$), or **demand** ($b(i) < 0$). The **minimum cost flow problem** can be stated as follows:

$$\text{minimize } z(x) = cx = \sum_{(i,j) \in E} c_{ij} x_{ij}$$

among all flows x .

For simplicity, we often use an algebraic notation for some functions, implicitly converting them to vectors, that is, b (without any brackets) or c (without any subscripts) can be used as vectors of values $b(i)$ for each node $i \in V$ and c_{ij} for each edge $(i, j) \in E$ respectively. The usage depends on the context.

Definition 1.4 (Pseudoflow [22]). A **pseudoflow** x is a function $x: E \rightarrow \mathbb{R}$ that satisfies only the non-negativity constraint $0 \leq x_{ij} \leq u_{ij}$ for all $(i, j) \in E$.

Clearly every flow x for any b is also a pseudoflow.

Definition 1.5 (Imbalance [22]). For a given pseudoflow $x: E \rightarrow \mathbb{R}$, we define the **imbalance** at node i to be

$$e(i) = b(i) + \sum_{\{j: (j,i) \in E\}} x_{ji} - \sum_{\{j: (i,j) \in E\}} x_{ij}.$$

A positive $e(i)$ is referred to as an *excess* and a negative $e(i)$ is called a *deficit*.

Definition 1.6 (Residual network [22]). The **residual network** for a graph $G = (V, E)$ with costs $c: E \rightarrow \mathbb{R}$, capacities $u: E \rightarrow \mathbb{R}$ and pseudoflow $x: E \rightarrow \mathbb{R}$ is denoted as $G(x) = (V, E')$ and defined as follows. We replace each arc $(i, j) \in E$ with two arcs $(i, j), (j, i) \in E'$. The arc (i, j) has cost c_{ij} and **residual capacity** $r_{ij} = u_{ij} - x_{ij}$ and the arc (j, i) has cost $-c_{ij}$ and residual capacity $r_{ji} = x_{ij}$. A residual network consists **only** of arcs with positive residual capacity.

1.3 Reduced costs

Definition 1.7 (Potentials and reduced costs [30]). For each node $i \in V$ we associate a real number $\pi(i)$ as the **potential** of node i . For a given set of node potentials π , we define the **reduced cost** of arc $(i, j) \in E$ as $c_{ij}^\pi = c_{ij} - \pi(i) + \pi(j)$

We show some useful correlations between networks with cost function c and networks with nodes associated with node potentials π and arc cost function c^π . Let $z(\pi) = \sum_{(i,j) \in E} c_{ij}^\pi x_{ij}$ denote an objective function for a minimum cost flow.

Lemma 1.1 (Property 2.4 in [30]). Minimum cost flow problems with arc costs c_{ij} and c_{ij}^π have the same optimal solutions. Moreover, $z(\pi) = z(0) - \pi b$.

Proof (see [30, p. 43]). Consider an initial node potential π_s . Let us define a new potential π'_s by increasing the potential at a specific node k by α . That is, $\pi'_s(k) = \pi_s(k) + \alpha$, and $\pi'_s(i) = \pi_s(i)$ for all $i \neq k$.

Recall the flow conservation constraint for any node i : $\sum_j x_{ij} - \sum_j x_{ji} = b(i)$. Thus evaluating the objective function under the new potential yields:

$$\begin{aligned} z(\pi'_s) &= \sum_{(i,j)} c_{ij}^{\pi'_s} x_{ij} = \sum_{(i,j)} c_{ij}^{\pi_s} x_{ij} - \sum_i \alpha x_{ki} + \sum_i \alpha x_{ik} \\ &= z(\pi_s) - \alpha b(k) \end{aligned}$$

Since $\alpha b(k)$ is a constant with respect to the flow variables x_{ij} , the optimal solution remains unchanged.

Therefore, if we start with a zero potential vector (0) and incrementally update it to reach an arbitrary potential vector π , we preserve optimality and obtain the equality $z(\pi) = z(0) - \sum_k \pi(k) \cdot b(k) = z(0) - \pi b$. $\square$

From the definition of reduced costs, the following lemma follows directly:

Lemma 1.2 (Property 9.2 in [30]). *The following properties occur*

1. For any directed path P from node k to node l , $\sum_{(i,j) \in P} c_{ij}^\pi = \sum_{(i,j) \in P} c_{ij} - \pi(k) + \pi(l)$
2. For any directed cycle W , $\sum_{(i,j) \in W} c_{ij}^\pi = \sum_{(i,j) \in W} c_{ij}$

1.4 Optimality conditions

Now we introduce the optimality conditions for residual networks, which will be used to prove the complementary slackness optimality conditions.

To prove optimality conditions theorem, let us first introduce the following theorem, which we state without proof.

Theorem 1.4.1 (Augmenting cycle theorem [30]). *Let x and x^* be any two feasible solutions of a network flow problem. Then x equals x^* plus the flow on at most m directed cycles in $G(x^*)$. Furthermore, the cost of x equals the cost of x^* plus the cost of flow on these augmenting cycles.*

Theorem 1.4.2 (Negative cycle optimality conditions [30]). *A feasible solution x^* is an optimal solution of the minimum cost flow problem if and only if it satisfies the negative cycle optimality conditions: namely, the residual network $G(x^*)$ contains no negative (directed) cycle.*

Proof (see [30, p. 307]). ($\Rightarrow$) Suppose that there $G(x)$ contains a negative-cost cycle. By pushing a positive flow along this cycle, we obtain a new feasible flow x' with a strictly lower cost. Therefore, x cannot be optimal.

($\Leftarrow$) Suppose that x is a feasible flow and the residual network $G(x)$ contains no negative-cost cycles. Let x^* be an optimal flow such that $x^* \neq x$. By Theorem 1.4.1 the flow difference $x^* - x$ can be decomposed to at most m augmenting cycles in $G(x)$. Due to the initial assumption, none of those cycles are negative-cost. Therefore, we have $c \cdot (x^* - x) \geq 0$, or equivalently $cx^* \geq cx$. Since x^* is an optimal have $cx^* \leq cx$. Thus $cx^* = cx$, which implies that x is also an optimal flow. $\square$

Theorem 1.4.3 (Reduced cost optimality conditions [30]). *A feasible solution x^* is an optimal solution of the minimum cost flow problem if and only if some set of node potentials π satisfy the following reduced cost optimality conditions:*

$$c_{ij}^\pi \geq 0 \quad \text{for every arc } (i, j) \text{ in } G(x^*).$$

Proof (see [30, p.309]). ($\Leftarrow$) Suppose that a feasible flow x^* satisfies $c_{ij}^\pi \geq 0$ for every arc (i, j) in $G(x^*)$ for some π . Then, for each directed cycle W in $G(x^*)$, we have $\sum_{(i,j) \in W} c_{ij}^\pi \geq 0$. By Lemma 1.2, we obtain $\sum_{(i,j) \in W} c_{ij}^\pi = \sum_{(i,j) \in W} c_{ij} \geq 0$, meaning $G(x^*)$ has no negative cycles. Therefore, by Theorem 1.4.2, x^* is optimal.

($\Rightarrow$) Assume that x^* is an optimal solution. By Theorem 1.4.2, $G(x^*)$ contains no negative cycles. Therefore, we can compute the shortest path distances from an arbitrary node s to all other nodes in $G(x^*)$, using c_{ij} as the weight for each arc (i, j) . Let $d(i)$ denote the distance from node s to node i . By the triangle inequality, we have $d(j) \leq d(i) + c_{ij}$ for all (i, j) in $G(x^*)$. Equivalently, this can be rewritten as $c_{ij} - (-d(i)) + (-d(j)) \geq 0$, which yields $c_{ij}^\pi \geq 0$ if we define $\pi = -d$. $\square$

1.4.1 Complementary slackness optimality conditions

We previously established the conditions for residual networks. Now, we will use those conditions to get the conditions for the original networks.

Theorem 1.4.4 (Complementary slackness optimality conditions [30]). *A feasible solution x^* is an optimal solution of minimal cost flow problem if and only if for some set of node potentials π , the reduced costs and flow values satisfy the following complementary slackness optimality conditions for every arc $(i, j) \in E$:*

$$\begin{aligned} c_{ij}^\pi > 0 &\Rightarrow x_{ij}^* = 0 \\ 0 < x_{ij}^* < u_{ij} &\Rightarrow c_{ij}^\pi = 0 \\ c_{ij}^\pi < 0 &\Rightarrow x_{ij}^* = u_{ij} \end{aligned}$$

Proof (see [30, p.310]). We show a proof of equivalence to the Theorem 1.4.3 in one direction for simplicity.

- Suppose that $c_{ij}^\pi > 0$ and $x_{ij}^* > 0$. Then, the residual network contains (j, i) . That contradicts Theorem 1.4.3, since $c_{ji}^\pi = -c_{ij}^\pi < 0$.
- Suppose that $0 < x_{ij}^* < u_{ij}$. The residual network contains both (i, j) and (j, i) . By Theorem 1.4.3, $c_{ij}^\pi \geq 0$ and $c_{ji}^\pi \geq 0$, thus $c_{ij}^\pi = c_{ji}^\pi = 0$.
- Suppose that $c_{ij}^\pi < 0$ and $x_{ij}^* < u_{ij}$. Then the residual network contains (i, j) . That contradicts Theorem 1.4.3, since $c_{ij}^\pi < 0$.

The converse is left to the reader as an exercise. $\square$

1.5 Dual problem

The minimum cost problem, as defined above, is a linear programming problem. For every problem of that kind we can associate another strongly related linear programming problem called its *dual*. The idea is based on the desire to limit the possible optimal values. For instance, the objective function

value of any feasible solution of the dual is less than or equal to the objective function of any feasible solution of the primal.

Sometimes dual problems are much more convenient to work with, than original, primal problems. It is a common pattern for algorithms solving minimum cost flow problem, including those shown in this work.

Upon forming the dual of a (primal) linear programming problem, each constraint of the primal, except for the non-negativity of arc flows, gets a corresponding *dual variable*. For the minimum cost flow problem, we have two dual variables, namely $\pi(i)$ and α_{ij} . The first one is associated with the mass balance constraint of node i and the second one corresponds to the capacity constraint of arc (i, j) .

Definition 1.8 (Dual minimum cost flow problem [30]). *Let $G = (V, E)$ be a directed network with a cost c_{ij} and a capacity u_{ij} associated with every arc $(i, j) \in E$. For each node $i \in V$, we have a number $b(i)$ which indicates its supply ($b(i) > 0$), or demand ($b(i) < 0$). The **dual minimum cost flow problem** can be stated as follows:*

$$\text{maximize } w(\pi, \alpha) = b\pi - u\alpha = \sum_{i \in V} b(i)\pi(i) - \sum_{(i,j) \in E} u_{ij}\alpha_{ij}$$

subject to

$$\begin{aligned} \pi(i) - \pi(j) - \alpha_{ij} &\leq c_{ij} && \text{for all } (i, j) \in E, \\ \alpha_{ij} &\geq 0 && \text{for all } (i, j) \in E, \\ \pi(j) &\text{unrestricted} && \text{for all } j \in V. \end{aligned}$$

In the two following theorems, we state that the primal and the dual are equivalent. The proofs may be found in [30, p.312–313].

Theorem 1.5.1 (Weak duality theorem [30]). *Let $z(x)$ denote the objective function value of some feasible solution x of the minimum cost flow problem and let $w(\pi, \alpha)$ denote the objective function value of some feasible solution (π, α) of its dual. Then $w(\pi, \alpha) \leq z(x)$.*

The first constraint of Definition 1.8 can be rewritten as $\alpha_{ij} \geq -c_{ij}^\pi$. We want to maximize the value of the objective function, so we want to assign the smallest possible value to α_{ij} . Thus $\alpha_{ij} = \max\{0, -c_{ij}^\pi\}$. Now we can rewrite the dual objective instead of $w(\pi, \alpha)$ as

$$w(\pi) = \sum_{i \in V} b(i)\pi(i) - \sum_{(i,j) \in E} \max\{0, -c_{ij}^\pi\}u_{ij}.$$

Theorem 1.5.2 (Strong duality theorem [30]). *For any choice of problem data, the minimum cost flow problem always has a solution x^* and the dual minimum cost flow problem has a solution π satisfying the property that $z(x^*) = w(\pi)$.*

This theorem shows that if the minimum cost flow problem has an optimal solution x^* , then it has a matching dual solution π satisfying the condition $z(x^*) = w(\pi)$. Moreover, the complementary slackness optimality conditions imply strong duality. The following theorem states the converse:

Theorem 1.5.3 (Theorem 9.7 in [30]). *If x is a feasible flow and π is an (arbitrary) vector satisfying the property that $z(x) = w(\pi)$, then the pair (x, π) satisfies the complementary slackness optimality conditions.*

1.6 Equivalent problems

Many algorithms do not solve the originally stated minimum cost flow problem, but rather similar problems that are equivalent to the original.

1.6.1 Minimum cost circulation with lower bounds

Let us introduce a minimum cost circulation. Some of the algorithms in the paper are tailored to exactly this kind of problem. We introduce a definition with flow lower bounds on the arcs. Let $G = (V, E)$ be a directed network with costs c_{ij} , capacities u_{ij} and lower-bounds l_{ij} for each arc $(i, j) \in E$.

Definition 1.9 (Circulation with lower bounds [31]). *A **circulation with lower bounds** $x : E \rightarrow \mathbb{R}^{\geq 0}$ is an assignment of nonnegative reals to the arcs such that the following two properties are obeyed:*

- for all edges $(i, j) \in E$,

$$l_{ij} \leq x_{ij} \leq u_{ij},$$
- for all $i \in V$, the total flow entering i is equal to the flow leaving i ; that is,

$$\sum_{k: (k, i) \in E} x_{ki} = \sum_{k: (i, k) \in E} x_{ik}.$$

The cost of the circulation is $c(x) = \sum_{(i, j) \in E} c_{ij} x_{ij}$.

Definition 1.10 (Minimum cost circulation with lower bounds [31]). *The **minimum cost circulation problem with lower bounds** can be stated as follows:*

$$\text{minimize } c(x) = \sum_{(i, j) \in E} c_{ij} x_{ij}$$

among all circulations with lower bounds.

Theorem 1.6.1. *The minimum cost flow problem and the minimum cost circulation problem are equivalent.*

Proof (see [31, p. 111]). ($\Rightarrow$) Given an instance of the minimum cost flow, we convert it to an instance of the minimum cost circulation problem. Add node s to the graph. For any $i \in V$ such that $b(i) > 0$, we create an arc (s, i) with $c_{si} = 0$ and $l_{si} = u_{si} = b(i)$. For any supply node $j \in V$ (with $b(j) < 0$), we create an arc (j, s) , with $l_{js} = u_{js} = -b(j)$ and cost $c_{js} = 0$. It is easy to see that the optimal costs are the same. The reduction is presented in Figure 2.

($\Leftarrow$) If we want to create a minimum cost flow instance from a circulation, we need to remove the lower-bounds of the flow from the arcs. To do this, we implicitly assume that the arc (i, j) already contains l_{ij} units of flow. Therefore, the imbalance $b(i)$ decreases by l_{ij} , and $b(j)$ increases by l_{ij} . In other words, we perform the following operations. Let $x'_{ij} = x_{ij} - l_{ij}$, and $u'_{ij} = u_{ij} - l_{ij}$. Then, set $b(i) = \sum_{k: (k,i) \in E} l_{ki} - \sum_{k: (i,k) \in E} l_{ik}$. Given a feasible circulation x in the original problem, we get a feasible flow x' for a minimum cost flow problem with the same costs, capacity u' and imbalance b . It is easy to see that the operations preserve optimality. $\square$

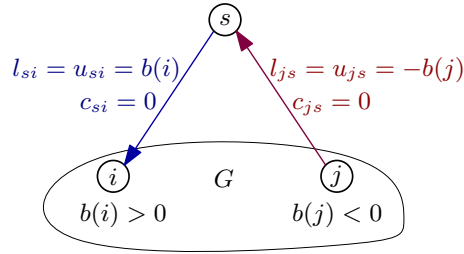


Figure 2: Reduction from minimum cost flow to minimum cost circulation problem.

1.6.2 Minimum cost circulation without lower bounds

Then we redefine the problem to remove lower bounds from the definition. It is more convenient to work with capacities only, and many algorithm descriptions for the minimum cost circulation problem uses this definition for simplicity.

Definition 1.11 (Circulation [31]). A **circulation** $x : E \rightarrow \mathbb{R}$ is an assignment of reals to the arcs such that the following three properties are obeyed:

- for all arcs $(i, j) \in E$,

$$x_{ij} \leq u_{ij},$$

- for all $i \in V$, the total flow leaving i is zero; that is,

$$\sum_{k: (i,k) \in E} x_{ik} = 0,$$

- for all $(i, j) \in E$

$$x_{ij} = -x_{ji}.$$

Definition 1.12 (Minimum cost circulation [31]). *The **minimum cost circulation** can be stated as follows:*

$$\text{minimize } c(x) = \frac{1}{2} \sum_{(i,j) \in E} c_{ij} x_{ij}$$

among all circulations.

Notice that the flow in the arc is taken twice (x_{ij} and x_{ji}), this is why we divide the cost in half.

We perform the reduction as follows. Let $G = (V, E)$ be the original network with lower-bounded arcs. We create a new network $G' = (V, E')$ that has no lower bounds, using only upper bounds u'_{ij} for each arc $(i, j) \in E'$.

For every original arc $(i, j) \in E$, we set the upper bound to $u'_{ij} = u_{ij}$. We then add a backward arc (j, i) with an upper bound of $u'_{ji} = -l_{ij}$.

By definition, flow is skew-symmetric, meaning $x'_{ij} = -x'_{ji}$. Because of this, if the backward flow is within its upper bound ($x'_{ji} \leq u'_{ji}$), we have $-x'_{ij} \leq -l_{ij}$. Multiplying both sides by -1 gives $x'_{ij} \geq l_{ij}$, which shows that the original lower bound requirement is met.

We have the similar optimality conditions for the minimum cost circulation as for minimum cost flow problem described in Section 1.4.

Theorem 1.6.2 (Theorem 5.3 in [31]). *Let $G(x)$ be a residual network for a graph G a circulation x . The following statements are equivalent for x :*

1. x is a minimum-cost circulation,
2. there is no negative-cost cycle in $G(x)$,
3. there are potentials π such that $c_{ij}^\pi \geq 0$ for all arcs (i, j) in $G(x)$.

1.6.3 Uncapacitated minimum cost flow

Since the arc capacities impose certain restrictions, there are algorithms designed to solve a simpler version of the minimum cost flow problem, with no capacities associated with the arcs. In this subsection, we present a well-known reduction from the capacitated minimum cost flow problem to the uncapacitated one.

Suppose that we have a capacitated network $G = (V, E)$. Let $G' = (V', E')$ be the new network with supply/demand b' , arc costs c' and no capacities. We perform the following reduction. Let $(i, j) \in E$ be an arc with capacity u_{ij} and cost c_{ij} . We introduce an additional node k and replace arc (i, j) with two arcs (i, k) and (j, k) as it is shown in Figure 3.

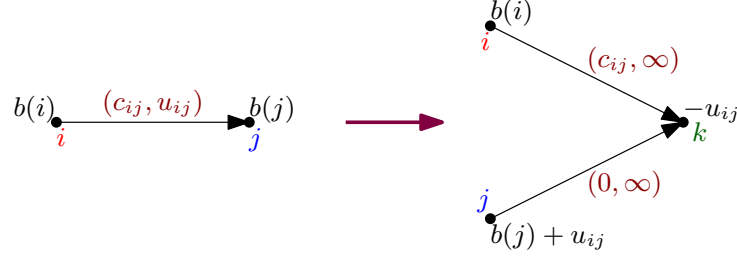


Figure 3: Reduction from capacitated to uncapacitated network.

Now we provide some justification of the reduction. Suppose that the flow x_{ij} on arc $(i, j) \in E$ is some arbitrary number $0 \leq a \leq u_{ij}$, and that $e(i) = b(i) - a$, $e(j) = b(j) + a$. Let $e' : V' \rightarrow \mathbb{R}$ denote an excess for uncapacitated network G' . We show that the excesses in the uncapacitated network match the original.

Initially we let $e'(i) := b(i)$ and $e'(j) := b(j)$. To get the same cost, we need to push a units of flow through the edge $(i, k) \in E'$. Now we have $e'(i) = b(i) - a$ and $e'(k) = -u_{ij} + a$. To enforce feasibility, the only option is to send $u_{ij} - a$ units of flow through the arc $(j, k) \in E'$. Therefore, we have equity $e(j) = b(j) + u_{ij} - (u_{ij} - a) = b(j) + a$ as in the capacitated case.

Chapter 2

Edmonds-Karp algorithm

In this chapter we present capacity scaling algorithm, introduced in [14]. We start from a simpler idea and then show how to improve the algorithm to achieve satisfying results. Both algorithms are described more widely in [30].

To begin with, certain assumptions must be made, which involve modifying the network. Below, we enumerate the assumptions and describe how to perform the corresponding network transformations.

Assumption 2.1. *All arc costs are nonnegative.*

If the network is capacitated, then we saturate the negative cost arcs. Otherwise, we need to set some node potentials and then apply the reduced costs. By Lemma 1.1 it preserves optimality. To find the right potentials, We follow the proof of Theorem 1.4.3. Let G' be a copy of the original graph G . For each arc (i, j) in G' , we assign a *weight* equal to the cost c_{ij} of the arc in the original. We compute the shortest path distances $d(i)$ from node s to all other nodes in the graph G' , using Bellman-Ford algorithm [32], for instance. If the shortest path algorithm identifies a negative cycle in network, then the minimum cost flow problem is infeasible. If not, we observe that $d(j) \leq c_{ij} + d(i)$, thus we replace c_{ij} by $c_{ij} + d(i) - d(j)$, which is nonnegative.

Assumption 2.2. *For each pair of nodes $i, j \in V$, there is a directed path from i to j in graph G .*

The above requirement can be easily met by adding an artificial node s with $b(s) = 0$ and adding artificial arcs (s, i) and (i, s) with a sufficiently high cost M .

We now introduce a lemma behind the key idea of the algorithms.

Lemma 2.1 (Lemma 9.11 in [30]). *Suppose that a pseudoflow x satisfies the reduced cost optimality conditions with respect to some node potentials π . Let the vector d represent the shortest path distances from some node s to all other nodes in the residual network $G(x)$ with c_{ij}^π as the length of an arc (i, j) . Then the following properties are valid:*

1. the pseudoflow x also satisfies the reduced cost optimality conditions with respect to the node potentials $\pi' = \pi - d$,
2. the reduced costs $c_{ij}^{\pi'}$ are zero for all arcs (i, j) in a shortest path from node s to every other node.

Proof. The first part is a straightforward generalization of Lemma 1.1 which can be proved by induction.

Let E' be the set of arcs $E' \subseteq E$ such that E' contains all arcs that are present in the shortest path tree with respect to distance vector d . Let us consider an arc $(i, j) \in E'$. Notice that $c_{ij}^{\pi} + d(i) = d(j)$, thus $c_{ij} - (\pi(i) - d(i)) + (\pi(j) - d(j)) = 0$, so it holds that $c_{ij}^{\pi'} = 0$. Therefore, the second property is valid. $\square$

The following result is an immediate corollary of the preceding lemma.

Lemma 2.2 (Lemma 9.12 in [30]). *Suppose that a pseudoflow x satisfies the reduced cost optimality conditions. Let x' be obtained from x by sending flow along a shortest path from node s to some other node k . Then x' also satisfies the reduced cost optimality conditions.*

2.1 Successive shortest path algorithm

First, we present a simplified algorithm, followed by some improvements. Algorithm 1 proceeds as follows:

1. Choose two nodes $k, l \in V$ such that $e(k) > 0$ and $e(l) < 0$.
2. Compute the shortest path from k to l using reduced costs as weights.
3. Update the potentials, as shown in Lemma 2.1.
4. Augment as much flow as possible along the path from k to l .

We repeat these steps until $e(i) = 0$ for every $i \in V$. The pseudocode is provided below.

Algorithm 1 SUCCESSIVESHORTESTPATHALGORITHM(G, b, c, u)

```

1:  $x \leftarrow 0^m, \pi \leftarrow 0^n$ 
2:  $e(i) \leftarrow b(i)$  for all  $i \in V$ 
3:  $S \leftarrow \{i: e(i) > 0\}, T \leftarrow \{i: e(i) < 0\}$ 
4: while  $S \neq \emptyset$  do
5:   select nodes  $k \in S$  and  $l \in T$   $\triangleright c^\pi$  from Definition 1.7
6:    $d \leftarrow \text{DISTANCES}(G(x), c^\pi, k)$   $\triangleright$  the weight of the arc  $(i, j)$  is  $c_{ij}^\pi$ 
7:    $P \leftarrow \text{SHORTESTPATH}(G(x), d, k, l)$   $\triangleright$  shortest path from  $k$  to  $l$ 
8:    $\pi \leftarrow \pi - d$ 
9:    $\delta \leftarrow \min\{e(k), -e(l), \min\{r_{ij}: (i, j) \in P\}\}$   $\triangleright r_{ij}$  from Definition 1.6
10:   $x \leftarrow \text{AUGMENT}(x, P, \delta)$ 
11:  update  $S, T$ 
12: end while
13: return  $x$ 

```

2.1.1 Correctness

We now show that during every step of the loop, the pseudoflow satisfies the reduced cost optimality condition given in Theorem 1.4.3. Due to Assumption 2.1, an initial empty flow meets these conditions.

No augmentation breaks flow feasibility, as the augmented flow value does not exceed the minimal residual capacity on the path. Lemma 2.2 ensures that the new flow also preserves the optimality conditions.

In each iteration, we decrease the excess on some node by at least one unit, gradually correcting the imbalance. Thus, the algorithm terminates when the imbalance at every node reaches zero. Since all necessary conditions are preserved throughout, the algorithm successfully finds an optimal solution.

2.1.2 Complexity

Let U denote an upper bound on the largest supply of the node and C denote an upper bound on the cost of the arc.

On every loop iteration, we augment at least one unit of flow. Therefore, each $k \in S$ can be chosen at most U times. Thus the algorithm terminates in at most nU iterations. Let $SP(n, m, C)$ denote the time taken to solve a shortest path problem in a graph with n nodes and m arcs, with nonnegative arc lengths bounded by some constant C . Since the reduced costs are bounded by nC , the running time of Algorithm 1 is $O(nU \cdot SP(n, m, nC))$.

2.2 Capacity scaling

Here, we introduce improvements to the previous algorithm using the *capacity scaling* technique. This approach allows us to reduce the number of augmentations and, consequently, the time complexity. The following discussion is based on the algorithm description in [30].

This technique introduces a new parameter Δ , initially set to $2^{\lceil \log U \rceil}$, where U is the upper bound for edge capacity ($U = \max_{ij \in E} \{u_{ij}\}$). The core idea is that the improved algorithm performs $\log U$ Δ -scaling phases, reducing Δ by a factor of 2 at the end of each phase.

During a Δ -scaling phase, we consider the Δ -residual network $G(x, \Delta)$, which is a subgraph of $G(x)$ containing only the arcs with a residual capacity r_{ij} greater than or equal to Δ .

Instead of augmenting as many units of flow as possible, we now send exactly Δ units during each augmentation in the Δ -scaling phase. We also only need to preserve the optimality conditions for the $G(x, \Delta)$ residual network.

For a given value of Δ , we define:

$$S(\Delta) = \{i: e(i) \geq \Delta\}, \quad T(\Delta) = \{i: e(i) \leq -\Delta\}.$$

The algorithm proceeds as follows:

1. Select $k \in S(\Delta)$ and $l \in T(\Delta)$.
2. Compute the shortest path from k to l in the residual network $G(x, \Delta)$ using reduced costs as weights.
3. Update the potentials, as shown in Lemma 2.1.
4. Augment Δ units of flow along the shortest path from k to l with respect to $G(x, \Delta)$.

While the optimality conditions for the Δ -residual network are preserved, they may be violated for the full residual network $G(x)$. Therefore, at the beginning of each Δ -scaling phase, we must saturate the arcs in $G(x, \Delta)$ that have negative reduced costs.

The pseudocode for the complete algorithm is presented in Algorithm 2.

Algorithm 2 CAPACITYSCALINGALGORITHM(G, b, c, u)

```

1:  $x \leftarrow 0^m, \pi \leftarrow 0^n$ 
2:  $e(i) \leftarrow b(i)$  for all  $i \in V$ 
3:  $\Delta \leftarrow 2^{\lfloor \log U \rfloor}$ 
4: while  $\Delta \geq 1$  do ▷  $\Delta$ -scaling phase
5:   for every arc  $(i, j)$  in residual graph  $G(x)$  do
6:     if  $r_{ij} \geq \Delta$  and  $c_{ij}^\pi < 0$  then ▷  $r_{ij}$  from Definition 1.6
7:        $x \leftarrow \text{AUGMENT}(x, (i, j), r_{ij})$  ▷ send  $r_{ij}$  units through arc  $(i, j)$ 
8:     end if
9:   end for
10:   $S(\Delta) \leftarrow \{i \in V : e(i) \geq \Delta\}$ 
11:   $T(\Delta) \leftarrow \{i \in V : e(i) \leq -\Delta\}$ 
12:  while  $S(\Delta) \neq \emptyset$  and  $T(\Delta) \neq \emptyset$  do
13:    select  $k \in S(\Delta)$  and  $l \in T(\Delta)$  ▷  $c^\pi$  from Definition 1.7
14:     $d \leftarrow \text{DISTANCES}(G(x, \Delta), c^\pi, k)$  ▷ the weight of the arc  $(i, j)$  is  $c_{ij}^\pi$ 
15:     $P \leftarrow \text{SHORTESTPATH}(G(x, \Delta), d, k, l)$  ▷ shortest path from  $k$  to  $l$ 
16:     $\pi \leftarrow \pi - d$ 
17:     $x \leftarrow \text{AUGMENT}(x, P, \Delta)$  ▷ send  $\Delta$  units through path  $P$ 
18:    update  $S(\Delta), T(\Delta)$ 
19:  end while
20:   $\Delta \leftarrow \Delta/2$ 
21: end while
22: return  $x$ 

```

2.2.1 Correctness

We now show that optimality for the Δ -residual network is preserved at the end of the Δ -scaling phase.

At the beginning of the phase, we saturate the arcs with negative reduced costs. Thus, we obtain a pseudoflow that preserves the optimality conditions for the Δ -residual network.

The augmentation loop is almost identical to the one in Algorithm 1. The main difference is that we always augment exactly Δ units of flow along the path. By Lemma 2.2, this new flow also preserves optimality. Therefore, at the end of the Δ -scaling phase, the reduced cost optimality conditions remain satisfied for $G(x, \Delta)$.

The algorithm successively reduces the parameter Δ by a factor of 2. Since we initially set Δ as a power of 2, we eventually reach $\Delta = 1$. Note that when $\Delta = 1$, $G(x, \Delta) = G(x)$. Therefore, the optimality conditions are ultimately preserved for the entire residual network. Moreover, because we always augment a positive amount of flow, the algorithm successfully reduces the node imbalances to 0 and terminates.

2.2.2 Complexity

Let us analyze the time complexity of the algorithm. There are $O(\log U)$ scaling phases. At the beginning of the Δ -scaling phase, there might be at most m arcs that do not preserve the optimality conditions, which we immediately saturate.

Each saturation can change the excess by at most 2Δ . Therefore, the sum of the excesses is bounded by $O((n+m)\Delta)$. Since each augmentation sends exactly Δ units of flow, there will be at most $O(n+m) = O(m)$ augmentations in each phase.

For each augmentation, we need to compute the shortest path. Thus, we have the following result:

Theorem 2.2.1 (Theorem 10.1 in [30]). *The capacity scaling algorithm solves the minimum cost flow problem in $O(m \log U \cdot SP(n, m, nC))$ time.*

Here, $SP(n, m, nC)$ represents the time needed to solve the shortest path problem for a network with nonnegative weights, n vertices, m arcs, and a maximum edge weight bounded by nC .

Using Fredman and Tarjan's implementation of Dijkstra's algorithm [33], we can compute the shortest path in $SP(n, m, nC) = O(m + n \log n)$ time to obtain the final computational complexity:

Theorem 2.2.2. *The capacity scaling algorithm solves the minimum cost flow problem in $O(m^2 \log U + nm \log U \log n)$ time.*

Chapter 3

Orlin algorithm

The Orlin's strongly polynomial algorithm [22] is a modification of the capacity scaling approach. Here we need the similar assumptions as in the previous algorithms.

Assumption 3.1. *All arc costs are nonnegative.*

Assumption 3.2. *For each pair of nodes $i, j \in V$, there is a directed path from i to j in graph G .*

These two assumptions are the same as Assumption 2.2 and Assumption 2.1. Here we need one additional assumption. We need to assert that the network is uncapacitated.

Assumption 3.3. *A given network is **uncapacitated** that is, the instance of a problem consists only of graph G and arc costs c .*

In Section 1.6.3 we present a reduction from a capacitated instance to an uncappeditated one. Since we modify the network by adding a node for each edge, the time complexity may be worse. In order to prevent this, we need to use a modified shortest path algorithm described in Section 3.1.3.

Algorithm 2 follows the capacity scaling approach for capacitated networks. Due to the modification, we get some useful properties.

Lemma 3.1 (Lemma 5 in [22]). *At every step of an uncappeditated version of the algorithm, the flow and the residual capacity of every arc in the residual network is an integral multiple of the scale factor Δ .*

This property allows us to eliminate the augmentations performed at line 7 of Algorithm 2.

3.1 Idea

Unlike the standard capacity scaling approach, this algorithm solves **the dual minimum cost flow problem**, as described in Definition 1.8. It first

computes the optimal node potentials and subsequently uses them to derive an optimal flow.

Several modifications to the traditional capacity scaling algorithm are required to achieve the desired result.

First, we introduce the concept of *contraction*. The intuition behind this is that if the flow x_{ij} across an arc (i, j) is sufficiently large, x_{ij} will remain positive until a feasible flow is found. Consequently, by complementary slackness (Theorem 1.4.4), the reduced cost will always satisfy $c_{ij}^\pi = 0$. This allows us to merge nodes i and j .

Next, we present a scenario where the capacity scaling algorithm with contraction fails to be strongly polynomial, and we introduce Orlin's approach to overcome this limitation.

Following this, the augmentation loop remains largely identical to that of the standard capacity scaling algorithm. However, if the capacitated minimum cost flow problem is reduced to an uncapacitated one, the shortest path computation can be optimized to improve the overall time complexity.

Finally, we must *uncontract* the nodes and compute the actual flow. Because the algorithm initially solves the dual problem, it yields only the optimal potentials π^* . Using these potentials, we construct an auxiliary network G' containing only the arcs (i, j) from G whose reduced costs $c_{ij}^{\pi^*}$ are exactly 0. According to Theorem 1.4.4, only these arcs can carry positive flow. Therefore, determining the optimal flow simply requires executing a maximum flow algorithm on G' .

We elaborate on these steps in detail below.

3.1.1 Contraction

In this section, we detail the contraction operation, which was originally introduced by Orlin [22]. Recall that we contract only those arcs guaranteed to carry positive flow in all subsequent phases. By Theorem 1.4.4, this contraction does not affect optimality.

The contraction operation proceeds as follows. Assuming we can compare node identifiers, let $i < j$ without loss of generality. For every arc (i, j) meeting the condition for positive flow, we *contract* node j into node i by performing the following substitutions:

- set $b(i) \leftarrow b(i) + b(j)$ and $e(i) \leftarrow e(i) + e(j)$,
- replace every arc (v, j) with the arc (v, i) ,
- replace every arc (j, v) with the arc (i, v) ,
- assign the new arcs the same costs as the arcs they replace,
- remove node j .

We must also determine a threshold flow value that guarantees the flow along an arc will remain positive until a feasible flow is found.

During a Δ -scaling phase, at most n augmentations can occur. Suppose an arc (i, j) is the residual arc augmented each time; its flow would change by at most $n\Delta$ units. Summing over all subsequent phases, the total flow change for any arc is bounded by $n(\Delta + \Delta/2 + \Delta/4 + \cdots + 1) < 2n\Delta$. Consequently, any arc $(i, j) \in E$ whose flow exceeds $2n\Delta$ during a Δ -scaling phase is guaranteed to maintain a positive flow in all subsequent phases. We define such arcs as *strongly feasible*.

Definition 3.1 (Strongly feasible arc [22]). *Let x_{ij} denote the flow across the arc $(i, j) \in E$ during a Δ -scaling phase. The arc (i, j) is called **strongly feasible** if $x_{ij} \geq 2n\Delta$.*

Because the contraction process itself may induce an additional augmentation, we establish the following lemma to adjust our bound.

Lemma 3.2 (Lemma 11 in [22]). *In a Δ -scaling phase, if $x_{ij} \geq 3n\Delta$ for some arc (i, j) in $\hat{E}$, then $x_{ij} > 0$ in all subsequent scaling phases.*

Proof (from [22, p. 346]). Without contractions, the flow change on any arc due to augmentations across all subsequent scaling phases is at most $n(\Delta + \Delta/2 + \Delta/4 + \cdots + 1)$. Each contraction causes at most one additional augmentation, and there are at most $n - 1$ contractions. Thus, the total flow change on any arc is strictly less than $3n\Delta$. $\square$

Next, we describe the *uncontraction* operation. Because computing the optimal flow requires the original network alongside the final node potentials, we must keep a record of every contraction. Furthermore, we must track how the potentials change across subsequent contractions to accurately compute the final potentials. Once the final potentials are obtained, we can calculate the reduced costs for each arc and proceed with the maximum flow algorithm, utilizing only the arcs that have a reduced cost of zero.

3.1.2 A difficult case for the algorithm with contraction

There is a specific scenario in which the time complexity of the algorithm might depend on $\log C$, where $C = \max\{|b(i)| : i \in V\}$. Consider the following pathological example provided by Orlin [22]:

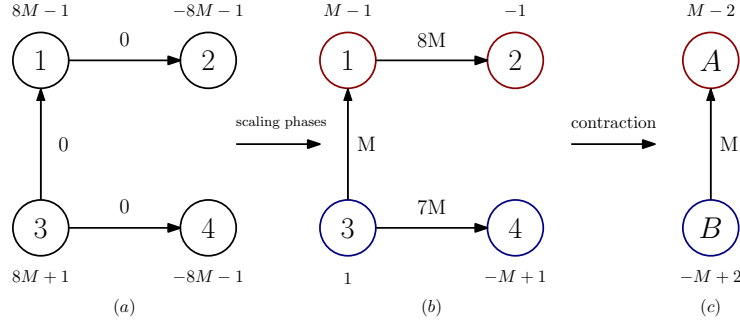


Figure 4: A hard case for the capacity scaling algorithm with contraction.
Figure adapted from [22].

Let M denote a sufficiently large number, specifically one that is exponentially larger than n and m . In Figure 4a, we present the initial supplies and demands. The algorithm with contractions performs the following sequence of augmentations:

- ($8M$ -scaling): AUGMENT($8M, 3 \rightarrow 2$),
- ($4M$ -scaling): AUGMENT($4M, 1 \rightarrow 4$),
- ($2M$ -scaling): AUGMENT($2M, 1 \rightarrow 4$),
- (M -scaling): AUGMENT($M, 1 \rightarrow 4$).

The resulting state is illustrated in Figure 4b. At this point, we can contract the arc $(1, 2)$, as shown in Figure 4c. However, achieving a feasible flow now requires $O(\log M)$ scaling phases. Consequently, the running time is no longer strongly polynomial.

This issue arises because it is possible to construct contracted nodes whose excess is close to Δ , even though their original supply or demand remains significantly smaller. Orlin observes that these problematic cases occur when the excess satisfies $e(k) = \Delta - \epsilon$ and $b(k) = -\epsilon$, or symmetrically, when $e(k) = -\Delta + \epsilon$ and $b(k) = \epsilon$, for some arbitrarily small $\epsilon > 0$ [22]. As a result, erasing the excess for such a node without further contractions requires a large number of augmentations to reach a feasible flow.

To overcome this difficulty, Orlin modifies the criteria for selecting active nodes for the sets $S(\Delta)$ and $T(\Delta)$. Specifically, a node u is included only if its excess satisfies $|e(u)| \geq \alpha\Delta$, for some constant $\frac{1}{2} < \alpha < 1$ [22]. Under this condition, we still augment Δ units of flow, but the total number of Δ -scaling phases becomes independent of the magnitude of the excesses.

Thus, in the strongly polynomial variant of the algorithm, we formally define the sets of active sources and sinks as $S(\Delta) = \{i \in V : e(i) \geq \alpha\Delta\}$ and $T(\Delta) = \{i \in V : e(i) \leq -\alpha\Delta\}$, respectively.

3.1.3 Optimized shortest path algorithm

As in the previous algorithm, we utilize Fredman and Tarjan's implementation of Dijkstra's algorithm [33] to compute the shortest paths. This implementation achieves a time complexity of $O(m + n \log n)$.

If the original network is capacitated, it must first be converted into an uncapacitated instance. We accomplish this using the reduction described in Section 1.6.3. However, this reduction introduces m additional nodes to the network, which inadvertently degrades the overall time complexity of the shortest path algorithm.

To overcome this issue, we can conceptually bypass these newly added vertices. For instance, we can identify them using a boolean flag. Observe that each added node k is adjacent to exactly two original nodes, say i and j . Note that, without loss of generality, if there is a residual arc from k to j , with residual capacity $r_{kj} \geq \Delta$, then, instead of considering residual arcs (i, k) and (k, j) , we can add one arc (i, j) to the shortest path graph. This operation is visualized in Figure 5. Thus we can effectively skip the intermediate node k during the main computation. By doing so, we maintain a graph with only $O(n)$ vertices for the shortest path algorithm. Once the shortest path distances for all original vertices have been determined, the distance for any added vertex k can be computed in $O(1)$ time using the relation $d(k) = \min\{d(i) + c_{ik}, d(j) + c_{jk}\}$.

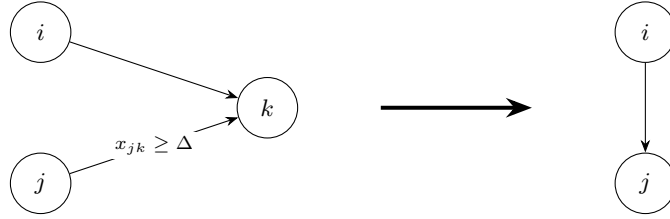


Figure 5: Graph nodes shortest path optimization.

3.2 Pseudocode

In this section we present a complete pseudocode, with the optimizations described in the previous section. In the description, we denote the contracted network by $\hat{G} = (\hat{V}, \hat{E})$.

Algorithm 3 STRONGLYPOLYNOMIALCAPACITYSCALING(G, b, c)

```

1:  $x \leftarrow 0^m, \pi \leftarrow 0^n$ 
2:  $e(i) \leftarrow b(i)$  for all  $i \in V$ 
3:  $\Delta \leftarrow 2^{\lceil \log(\max\{|e(i)| : i \in V\}) \rceil}$ 
4: while there is a node  $i \in \hat{V}$  with  $e(i) \neq 0$  do
5:   if  $x_{ij} = 0$  for each  $(i, j) \in \hat{E}$  and  $2^{\lceil \log|e(i)| \rceil} < \Delta$  for each  $i \in \hat{V}$  then
6:      $\Delta \leftarrow 2^{\lceil \log(\max\{|e(i)| : i \in V\}) \rceil}$ 
7:   end if
8:   while there is an arc  $(i, j) \in \hat{E}$  with  $x_{ij} \geq 3n\Delta$  do  $\triangleright \Delta$ -scaling phase
9:      $c_{kl} \leftarrow c_{kl}^\pi$  for each arc  $(k, l) \in \hat{E}$ 
10:     $\hat{G} \leftarrow \text{CONTRACT}(\hat{G}, i, j)$ 
11:  end while
12:   $S(\Delta) \leftarrow \{i \in \hat{V} : e(i) \geq \alpha\Delta\}$ 
13:   $T(\Delta) \leftarrow \{i \in \hat{V} : e(i) \leq -\alpha\Delta\}$ 
14:  while  $S(\Delta) \neq \emptyset$  and  $T(\Delta) \neq \emptyset$  do
15:    select  $k \in S(\Delta)$  and  $l \in T(\Delta)$   $\triangleright c^\pi$  from Definition 1.7
16:     $d \leftarrow \text{DISTANCES}(G(x, \Delta), c^\pi, k)$   $\triangleright$  the weight of the arc  $(i, j)$  is  $c_{ij}^\pi$ 
17:     $P \leftarrow \text{SHORTESTPATH}(\hat{G}(x, \Delta), d, k, l)$   $\triangleright$  shortest path from  $k$  to  $l$ 
18:     $\pi \leftarrow \pi - d$ 
19:     $x \leftarrow \text{AUGMENT}(x, P, \Delta)$   $\triangleright$  send  $\Delta$  units through path  $P$ 
20:    update  $S(\Delta), T(\Delta)$ 
21:  end while
22:   $\Delta \leftarrow \Delta/2$ 
23: end while
24:  $\text{UNCONTRACT}(\hat{G}, x)$ 
25: return  $x$ 

```

3.3 Correctness and complexity of the algorithm

3.3.1 Correctness

We now show the correctness of the algorithm.

Lemma 3.3 (Lemma 9 in [22]). *At every step of the strongly polynomial time algorithm, the flow and the residual capacity of each arc (i, j) in the residual network is an integral multiple of the scale factor Δ .*

Proof. Recall that Δ is a power of 2. At each consecutive phase, we divide Δ by two. Since we always send Δ units of flow, the flow value for each arc is divisible by Δ . The network is uncapacitated, thus for any residual arc (i, j) , the residual capacity r_{ij} , is either $r_{ij} = \infty$, or $r_{ij} = x_{ji}$. $\square$

This lemma ensures that each augmentation can send exactly Δ units of flow. Throughout its execution, the algorithm maintains a pseudoflow that satisfies the optimality conditions, and it terminates once we get a feasible flow.

At the end of each Δ -scaling phase, the reduced cost optimality conditions remain satisfied for $G(x, \Delta)$. The further analysis is similar to the one from Algorithm 2.

3.3.2 Complexity

Let Δ' and Δ denote the scaling factors in two consecutive phases, with $\Delta' \geq 2\Delta$ (the inequality may be strict, if the condition in the line 5 of Algorithm 3 holds). We say that a node k is *regenerated* if, at the start of the Δ -scaling phase, its excess satisfies $\alpha\Delta \leq |e(k)| < \alpha\Delta'$. If we contract some pair of nodes during the Δ -scaling phase, into one node k , we do not consider the node k as regenerated in that phase [22].

The following lemma provides an upper bound on the number of augmentations that can occur during the Δ -scaling phase. The proof is complicated, so we decided not to quote it, but it can be found in [22].

Lemma 3.4 (Lemma 10 in [22]). *The number of augmentations during the Δ -scaling phase is bounded by the number of regenerated nodes plus the number of contractions in that phase.*

Lemma 3.5 (Lemma 12 in [22]). *At each stage of the algorithm, $e(k) = b(k) \bmod \Delta$ for every node $k \in \hat{V}$.*

Proof (see [22, p. 346]). The value $e(k)$ is $b(k)$ minus the flow of each outgoing residual arc. By Lemma 3.3, each arc flow is a multiple of Δ . Thus we have $e(k) = b(k) \bmod \Delta$. $\square$

Lemma 3.6 (Lemma 13 in [22]). *The first time that a node k is regenerated, it satisfies $|e(k)| \leq 2\alpha|b(k)|/(2 - 2\alpha)$.*

Due to the long and tedious calculations involved, the proof of this lemma has been omitted.

We need to introduce a lemma proving the idea of the contraction.

Lemma 3.7 (Lemma 8 in [22]). *Suppose that at the termination of the Δ -scaling phase, $|b(i)| > 5n^2\Delta$ for some node i in V . Then there is an arc (i, j) or (j, i) incident to node i such that the flow on the arc is at least $3n\Delta$.*

Proof (see [22, p.344]). Let $i \in V$ satisfy $b(i) > 5n^2\Delta$ (for $b(i) < -5n^2\Delta$, the proof is analogous). Note that after the Δ -scaling phase, the total bound for excess or deficit cannot be greater than $2n\Delta$. Thus, we have $e(i) \leq 2n\Delta$. The number of adjacent nodes to i is less than n . Therefore, we bound the value of flow between any adjacent node by $(b(i) - e(i))/n \geq (5n^2\Delta - 2n\Delta)/n \geq 3n\Delta$. $\square$

Lemma 3.8 (Lemma 14 in [22]). *Each node is regenerated $O(\log n)$ times.*

Proof (from [22, p. 347]). Suppose that a node k is regenerated for the first time at the Δ^* -scaling phase. Let $\alpha^* = 2\alpha/(2 - 2\alpha)$. Then $\alpha\Delta^* \leq |e(k)| \leq \alpha^*|b(k)|$, where the second inequality follows from Lemma 3.6. After $\lceil \log(5\alpha^*n^2/\alpha) \rceil =$

$O(\log n)$ scaling phases, the scale factor is at most $\Delta^*/5\alpha^*n^2 \leq |b(k)|/5n^2$. By Lemma 3.7, there exists a strongly feasible arc emanating from node k . The node k then contracts into a new node and is (vacuously) not regenerated again. $\square$

Since there are at most n nodes in the contracted network, the following theorem is immediate.

Theorem 3.3.1 (Theorem 2 in [22]). *The total number of augmentations over all scaling phases is $O(n \log n)$.*

Next theorem bounds the number of scaling phases performed by algorithm. We state the theorem without a proof. Since we know by Theorem 3.3.1 that the number of scaling phases with at least one augmentation is $O(n \log n)$, it must be proven that the bound on the number of scaling phases without any augmentation is also $O(n \log n)$. This proof is more technical, and may be found in [22]. Having the bounds, the following theorem is immediate.

Theorem 3.3.2 (Theorem 3 in [22]). *The algorithm performs $O(n \log n)$ scaling phases.*

Theorem 3.3.3 (Theorem 4 in [22]). *The strongly polynomial algorithm determines the minimum cost flow in the contracted network in $O((n \log n)SP(n, m))$ time.*

Proof (from [22, p. 347]). Let us overview the time complexity of the operations in Δ -scaling phase. Reducing the scale factor by a factor other than two requires $O(m)$ time. Identification of the contractible arcs can also be done in $O(m)$ time. We identify the sets $S(\Delta)$ and $T(\Delta)$ in $O(n)$ running time. By Theorem 3.3.2 there are $O(n \log n)$ scaling phases, so these operations require $O((n \log n)m)$ total time. Each augmentation involves solving a shortest path problem and augmenting flow along this time. So the total time needed to perform these operations is $O((n \log n)SP(n, m))$ $\square$

The last theorem applies for the capacitated networks using the shortest path algorithm approach given in Section 3.1.3. Recall that for the capacitated network, we need to add an additional node for each arc to transform it into an uncapacitated network. Those modifications explain the difference between time complexities.

Theorem 3.3.4 (Theorem 5 in [22]). *The strongly polynomial algorithm from [22] solves the uncapacitated minimum cost flow problem in $O(n \log n(m + n \log n))$ time, and the capacitated minimum cost flow problem in $O(m \log n(m + n \log n))$ time.*

Chapter 4

Goldberg-Tarjan algorithm

In this chapter, we present a successive approximation algorithm developed by Goldberg and Tarjan [21] to solve the minimum-cost circulation problem defined in Section 1.6.2. Unlike the two previous algorithms, this approach employs a distinct technique known as *cost scaling*. Fundamentally, it builds upon the *push-relabel* framework, a well-established method for network flow problems.

4.1 Definitions

First, let us introduce some important terms. Recall that by Theorem 1.4.4 we have $c_{ij}^\pi < 0 \Rightarrow x_{ij} = u_{ij}$. The following definition adds a constant $\epsilon \geq 0$ and make this constraint slightly weaker.

Definition 4.1 (ϵ -optimality [21]). *For a constant $\epsilon \geq 0$, a pseudoflow x is said to be **ϵ -optimal with respect to the potential π** if for every arc $(i, j) \in E$, we have*

$$c_{ij}^\pi < -\epsilon \Rightarrow x_{ij} = u_{ij}$$

*A pseudoflow x is said to be **ϵ -optimal** if it is ϵ -optimal with respect to some potential π .*

The following lemma is immediate:

Lemma 4.1. *A pseudoflow x is ϵ -optimal with respect to a potential π if and only if $c_{ij}^\pi > -\epsilon$ for every residual arc (i, j) .*

The following theorem gives that if a circulation x is ϵ -optimal and the ϵ is sufficiently small, then the circulation x is optimal.

Theorem 4.1.1 (Theorem 2.3 in [21]). *If all costs are integers and $\epsilon < 1/n$, then an ϵ -optimal circulation x is minimum-cost.*

4.2 The algorithm

This section outlines the core concepts of the algorithm and its components, beginning with the successive approximation framework. Let $C = \max\{|c_{ij}| : (i, j) \in E\}$ denote the maximum possible cost in the considered network.

4.2.1 The successive approximation framework

The general procedure is outlined below.

Initially, the potentials π and the flow x are set to zero. To enforce ϵ -optimality, we start with a sufficiently large ϵ that is at least as great as all arc costs in the network, i.e., $\epsilon = C$.

We then enter a loop. In each iteration, while $\epsilon > 1/n$, we halve ϵ and calculate the new ϵ -optimal circulation x and its potential π . Once $\epsilon < 1/n$, the optimal solution is found, according to Theorem 4.1.1.

The REFINE operation is described later. To implement it, we will reuse several concepts established in the maximum flow problem.

Algorithm 4 SUCCESSIVEAPPROXIMATIONALGORITHM(G, c, u)

```

1:  $x \leftarrow 0^m$ 
2:  $\pi \leftarrow 0^n$ 
3:  $\epsilon \leftarrow C$ 
4: while  $\epsilon > 1/n$  do
5:    $\epsilon \leftarrow \epsilon/2$ 
6:    $x, \pi \leftarrow \text{REFINE}(x, \pi, \epsilon)$ 
7: end while
8: return  $x$ 
```

4.3 Push-relabel

Here we present the RELABEL implementation for calculating an ϵ -optimal pseudoflow. More specifically, given a 2ϵ -optimal circulation, we aim to obtain an ϵ -optimal one. This method is based on the well-known PUSHRELABEL technique for network flows.

To create an ϵ -optimal pseudoflow, we only need to saturate all residual arcs (i, j) satisfying $c_{ij} < -\epsilon$. The authors actually use a stronger operation, saturating every residual arc $(i, j) \in E$ that satisfies $c_{ij}^\pi < 0$.

The next step is to apply push operations to make the circulation feasible; i.e., we must push the excess out of all nodes to enforce the flow conservation constraints. In other words, we repeat the PUSHRELABEL operation while $e(u) > 0$ for some $u \in V$.

Algorithm 5 Push-relabel version of REFINES

```

1: procedure PUSHRELABEL( $i$ )
2:   if exists  $j \in V$  such that  $r_{ij} > 0$  and  $c_{ij}^\pi < 0$  then
3:     PUSH( $(i, j)$ )
4:   else
5:     RELABEL( $i$ )
6:   end if
7: end procedure
8:
9: procedure REFINES( $x, \pi, \epsilon$ )
10:  for  $(i, j) \in E$  such that  $c_{ij}^\pi < 0$  do
11:     $x_{ij} \leftarrow u_{ij}$   $\triangleright$  we push maximum flow, no matter the excess
12:  end for
13:  while exists  $i \in V$  such that  $e(i) > 0$  do
14:    PUSHRELABEL( $i$ )
15:  end while
16:  return  $x, \pi$ 
17: end procedure

```

We now describe how the PUSHRELABEL operation works. Given a node u , the operation attempts to push flow out of the node if possible; specifically, along any residual arc (u, v) where the residual capacity $r_{uv} = u_{uv} - x_{uv} > 0$ and the reduced cost c_{uv}^π is negative. If no arc satisfies these conditions, we must update the potential of node u .

The pseudocode for the PUSH and RELABEL operations is provided below.

Algorithm 6 PUSH and RELABEL procedures

```

1: procedure PUSH( $i, j$ )
2:    $\delta \leftarrow \min\{e(i), r_{ij}\}$ 
3:    $x_{ij} \leftarrow x_{ij} + \delta$ 
4:    $x_{ji} \leftarrow x_{ji} - \delta$ 
5: end procedure
6:
7: procedure RELABEL( $i$ )
8:    $\pi(i) \leftarrow \max_{(i,j) \in E(x)} \{\pi(j) + c_{ij} + \epsilon\}$ 
9: end procedure

```

We now describe the PUSH and RELABEL methods:

1. PUSH: If there exists a pair of nodes $i, j \in V$ where i has a positive excess $e(i) > 0$, and an arc (i, j) has a negative reduced cost and positive residual capacity, then we send $\min\{e(i), r_{ij}\}$ units of flow through the arc (i, j) .
2. RELABEL: If no push operations are possible, select a node $i \in V$ such that $e(i) > 0$ and update its potential $\pi(i)$ to $\max_{(i,j) \in E(x)} \{\pi(j) + c_{ij} + \epsilon\}$, where

$E(x)$ is the set of arcs in the residual graph $G(x)$. Note that this preserves ϵ -optimality, since for any arc $(i, j) \in E(x)$, $c_{ij}^\pi = c_{ij} - \pi(i) + \pi(j) \geq -\epsilon$.

Notice that the relabel operation on node $i \in V$ decreases $\pi(i)$ by at least ϵ , since all outgoing arcs (i, j) in $G(x)$ previously had non-negative reduced costs $c_{ij}^\pi \geq 0$. Therefore, if any incoming arc $(k, i) \in E(x)$ satisfied $c_{ki}^\pi \geq -\epsilon$ before the relabeling, it must subsequently satisfy $c_{ki}^\pi \geq 0$. Thus, there are no admissible arcs entering i [31].

4.3.1 Complexity

Next we analyze the complexity of the particular components in the successive approximation algorithm. Most of the results are left without a proof in this work, but the proofs can be found in the book [31]. The first theorem bounds the number of iterations of the loop inside the algorithm framework.

Theorem 4.3.1 (Theorem 5.35 in [31]). *Algorithm 4 takes $O(\log(nC))$ iterations to compute a minimum cost circulation.*

Now we focus on the push-relabel refine implementation.

Lemma 4.2 (Lemma 5.39 in [31]). *For any $i \in V$, $\pi(i)$ decreases by at most $3n\epsilon$ during the execution of the subroutine in Algorithm 5.*

Lemma 4.3 (Lemma 5.40 in [31]). *The number of relabel operations is $O(n^2)$.*

Proof (from [31, p. 138]). In each relabel operation of $i \in V$, we decrease $\pi(i)$ by at least ϵ . Since by Lemma 4.2 we can decrease $\pi(i)$ by at most $3n\epsilon$ overall, we perform at most $3n$ relabel operations on i . Since there are n nodes in total, we perform at most $3n^2$ relabel operations. $\square$

We now distinguish between saturating pushes ($\delta = r_{ij}$ units of flow on (i, j)) and nonsaturating pushes ($\delta = e(i) < r_{ij}$ units of flow on (i, j)). We can bound the number those pushes.

Lemma 4.4 (Lemma 5.41 in [31]). *The number of saturating push operations is $O(mn)$.*

Proof (from [31, p. 139]). Pick any (i, j) . Initially $c_{ij}^\pi \geq 0$, and so we need to relabel i before we can perform a saturating push on (i, j) . In order to make another push on this arc, we need to push flow through (j, i) . After that, we see that $c_{ij}^\pi \geq 0$. Therefore, i needs to be relabeled again. We see that before each saturating push, i must be relabeled. Thus by Lemma 4.2 for one arc we can perform at most $3n$ saturating pushes, and there are at most $3mn$ saturating pushes overall. $\square$

We present only the result of the nonsaturating push operations bound, again provided without a proof.

Lemma 4.5 (Lemma 5.43 in [31]). *The number of nonsaturating push operations is $O(mn^2)$.*

Putting everything together, we can show that the subroutine in Algorithm 5 can be implemented in $O(mn^2)$ time. Finally, we can compute the running time complexity of the Algorithm 4. We have the following theorem:

Theorem 4.3.2. *Algorithm 4 finds a minimum-cost circulation in $O(mn^2 \log(nC))$ time.*

4.4 The wave method

To develop a faster version of REFINE, we must impose a specific order on the PUSH and RELABEL operations. In this section, we present the WAVE method.

First, we introduce a *discharge* operation, as described in Algorithm 7. This procedure repeatedly applies PUSH and RELABEL operations to a given node until it is relabeled or its excess is reduced to zero.

Algorithm 7 DISCHARGE

```

1: procedure DISCHARGE( $v$ )
2:   while  $v$  is not relabeled and  $e(v) > 0$  do
3:     PUSHRELABEL( $v$ )
4:   end while
5: end procedure

```

To present the reordering approach, we need the following lemma.

Lemma 4.6 (Lemma 5.42 in [31]). *The set of arcs (i, j) satisfying $r_{ij} > 0$ and $c_{ij}^\pi < 0$ is acyclic.*

Definition 4.2 (Topological order). *Let $G = (V, E)$ be a directed acyclic graph. Let $\omega(u)$ denote the position of node u in an ordering ω of all nodes in V . The ordering is **topological** if and only if for every arc $(i, j) \in E$, the condition $\omega(i) < \omega(j)$ is satisfied.*

Let $G(x, \pi) = (V, E(x, \pi))$ be the graph constructed from the residual graph $G(x)$, containing only those arcs that have negative reduced costs. More formally, every arc $(i, j) \in E(x, \pi)$ must satisfy $u_{ij} - x_{ij} > 0$ and $c_{ij}^\pi < 0$. The wave method maintains a list of nodes L , topologically sorted with respect to the graph $G(x, \pi)$. By Lemma 4.6, the network $G(x, \pi)$ is acyclic, ensuring a valid topological order exists.

We subsequently discharge all nodes that have a positive excess. Note that if a node u has been relabeled, there are no residual arcs with negative reduced costs entering u . Therefore, we can safely move node u to the front of list L .

The algorithm continues this refinement process until the excess of all nodes is 0.

Algorithm 8 WAVE

```

1: procedure WAVE( $x, \pi, \epsilon$ )
2:    $L \leftarrow$  list of all nodes.
3:    $v \leftarrow L.FIRST(L)$   $\triangleright v$  is the current vertex of  $L$ 
4:   while there is  $i \in V$  such that  $e(i) > 0$  do
5:     if  $e(v) > 0$  then
6:       DISCHARGE( $v$ )
7:       if DISCHARGE relabeled  $v$  then
8:          $L.MOVETOFRONT(v)$ 
9:       end if
10:    end if
11:     $v \leftarrow L.NEXT(v)$   $\triangleright$  next in  $L$ , with respect to the old position of  $v$ 
12:  end while
13: end procedure

```

4.4.1 Complexity

We now analyze the time complexity of the wave implementation of refine.

Lemma 4.7 (Lemma 6.4 in [21]). *The wave method terminates after $O(n^2)$ passes through L .*

Proof (from [21, p. 27]). Lemma 4.6 implies that the wave method maintains the invariant that L is topologically ordered. If a pass over L occurs during which no relabelings occur, then the ordering of L does not change during the pass and every vertex is able to reduce its excess to zero. Thus no active vertices exists after the pass. $\square$

Theorem 4.4.1 (see Theorem 6.5 in [21]). *The wave implementation of FIND-EPSILONOPTIMALCIRCULATION runs in $O(n^3)$ time, giving an $O(n^3 \log(nC))$ bound for finding a minimum-cost circulation.*

Proof (from [21, p. 28]). Lemma 4.7 implies that there are $O(n^3)$ nonsaturating pushes (one per vertex per pass) during an execution of the procedure. By Lemma 4.4 there are $O(mn)$ saturating pushes. The time taken by the procedure is $O(n^3)$ plus $O(1)$ per push, for a total of $O(n^3)$. $\square$

Chapter 5

Benchmarks

In this chapter, we present the running times of the algorithms. We conducted benchmarks based on well-known instance generators, including Netgen [34] and Gridgen [35].

The implementations of the algorithms follow the descriptions provided in previous chapters. For Orlin's algorithm, due to the network transformation into an uncapacitated form, we apply a specific heuristic: it iterates over all nodes added during the transformation and pushes Δ units of flow along the zero-cost edges. This operation significantly improves the running time.

5.1 Running times and graphs

In this section, we present the running times of the algorithms for instances generated by Netgen and Gridgen. Let n denote the number of nodes and m the number of arcs in the network. The test cases are divided into the following groups:

1. 8-density ($m = 8n$),
2. $\sqrt{n}$ -density ($m = n\sqrt{n}$),
3. increasing arc number (increasing m with a constant n),
4. increasing arc capacities (with $m = 8n$),
5. increasing arc costs (with $m = 8n$),
6. increasing source and target nodes (with $m = 8n$),
7. a TCT benchmark [36].

All test cases were executed on a machine equipped with an Intel Core i7-6850K processor running at 3.2 GHz.

5.1.1 8-density

This subsection presents the running times for test cases where the number of arcs is fixed at $m = 8n$. Figure 6 illustrates the results for instances generated by Netgen and Gridgen.

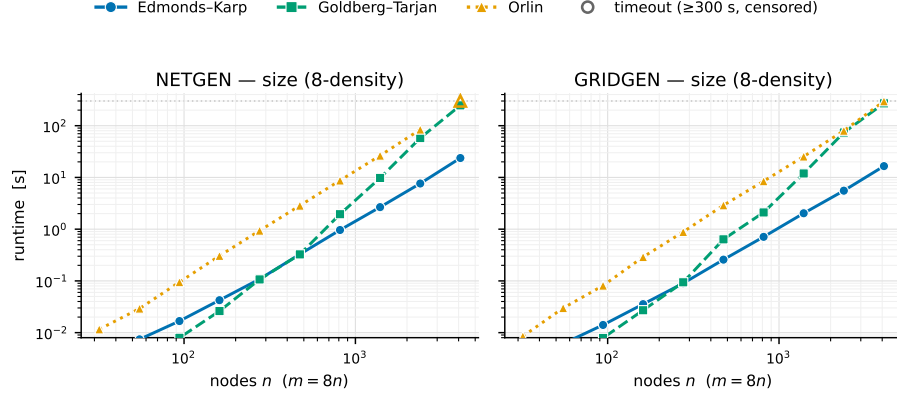


Figure 6: Running time graphs for test cases satisfying $m = 8n$.

5.1.2 $\sqrt{n}$ -density

This subsection presents the running times for test cases where the number of arcs satisfies $m = n\sqrt{n}$. The results are illustrated in Figure 7.

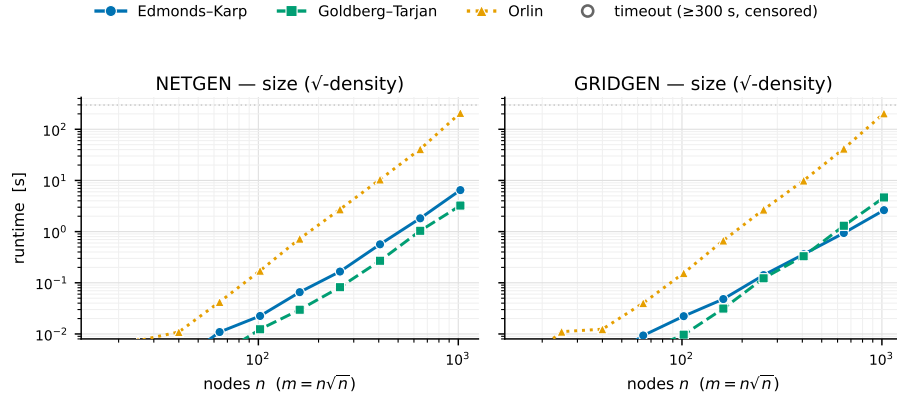


Figure 7: Running time graphs for test cases satisfying $m = n\sqrt{n}$.

5.1.3 Increasing arc number

In this subsection, we present the running times for test cases where the number of nodes is fixed at $n = 1024$, while the number of arcs increases. Notably, the implementation of Goldberg and Tarjan’s algorithm is generally faster for instances with a higher number of arcs. The running times of the remaining algorithms increase as expected. These results are illustrated in Figure 8.

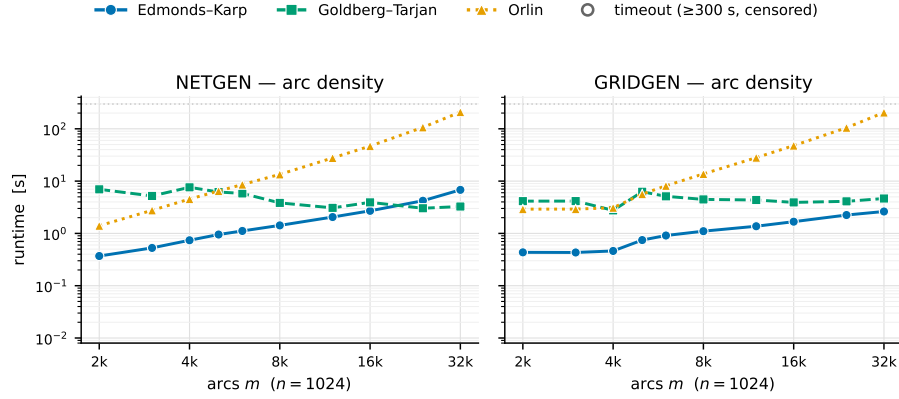


Figure 8: Running time graphs for a constant number of nodes n and an increasing number of arcs m .

5.1.4 Increasing capacities

This subsection details the running times for test cases with increasing arc capacity bounds. It may be surprising that the Edmonds-Karp algorithm performs faster on the final two tests; this occurs because the arc capacities exceed the total supply and demand at the nodes. The running times are illustrated in Figure 9.

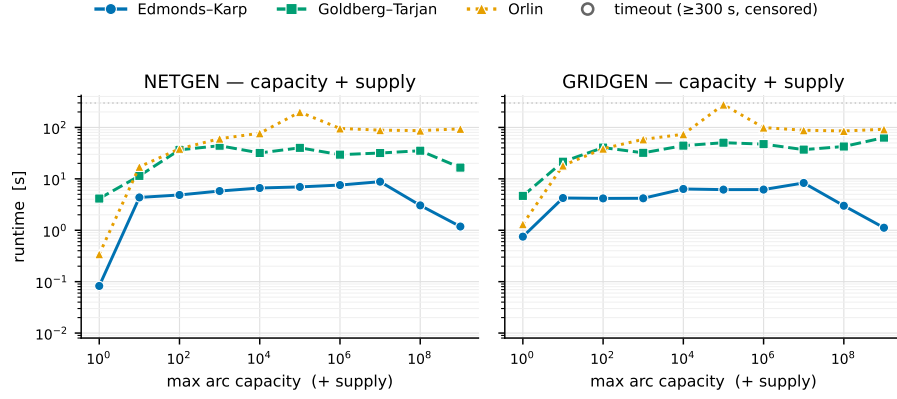


Figure 9: Running time graphs for varying arc capacity bounds and increasing supply/demand among all vertices.

5.1.5 Increasing costs

Here, we present the running times for test cases with increasing arc cost bounds. The running time of Goldberg and Tarjan's algorithm increases alongside the maximum arc cost. Because the time complexity of this algorithm depends on the cost, this behavior is entirely expected. The other algorithms exhibit relatively unchanged running times across all test instances, as their performance does not strictly depend on arc costs. These results are illustrated in Figure 10.

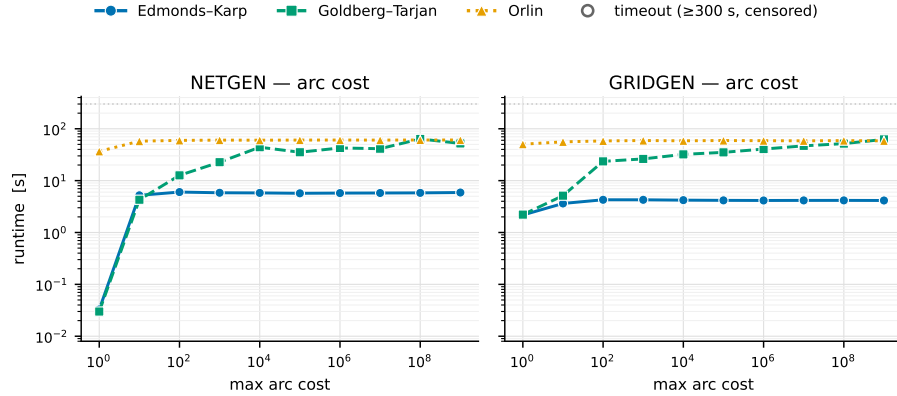


Figure 10: Running time graphs for increasing arc cost bounds.

5.1.6 Increasing source and target nodes

This subsection presents the running times for test cases where the number of supply and demand nodes increases. The motivation for this benchmark stems

from the observation that Orlin’s algorithm can be slower due to the increased number of vertices and supply/demand nodes resulting from the uncapacitated network transformation. We aimed to determine whether a large number of supply or demand nodes would cause a significant difference in performance. However, as illustrated in Figure 11, the running times remain relatively stable without major variations.

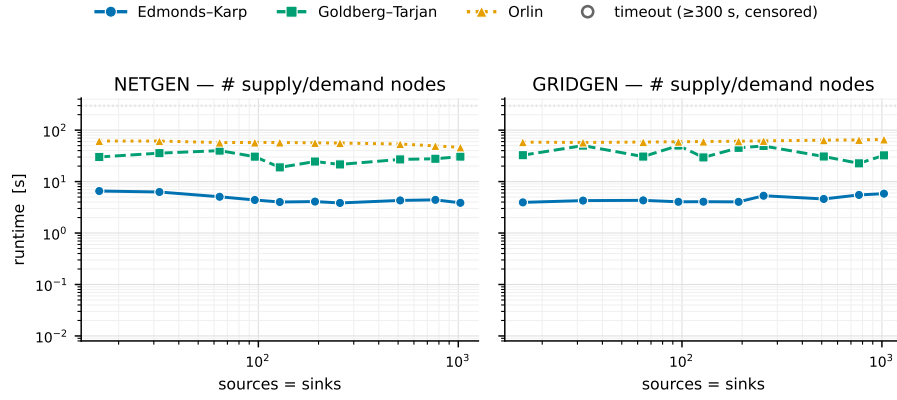


Figure 11: Running time graphs for an increasing number of source and target nodes.

5.1.7 TCT benchmark

The TCT benchmarks represent a linear relaxation of the time-cost tradeoff problem in *VLSI threshold voltage assignment*. It is a design technique that assigns voltages to transistors, minimizing the static leakage power without violating timing constraints. This problem can be modeled as a minimum cost flow problem.

The following test cases are taken from *Large Benchmarks for the Minimum-Cost Flow Problem* [36].

Table 4: TCT Benchmark Instances: Graph Properties

Testcase File	Nodes (V)	Arcs (E)
TCTS.dimacs	5,947	14,907
TCTL.dimacs	623,632	1,234,893
TCTB.dimacs	936,216	2,114,029
TCTD.dimacs	1,650,760	3,491,477
TCTM.dimacs	3,393,802	7,528,057
TCTV.dimacs	11,506,694	24,544,169

The results are shown in Table 5. Goldberg and Tarjan’s algorithm is far

better for these instances than the other tested algorithms. The reason for the significant time difference is the different algorithmic approach. The capacity scaling algorithm is highly inefficient in this case.

Table 5: TCT MCF Benchmark Results: CPU Time (Limit: 3h)

Testcase	Edmonds-Karp (s)	Orlin (s)	Goldberg-Tarjan (s)
TCTS.dimacs	3.450	6.268	0.018
TCTL.dimacs	TLE	TLE	17.452
TCTB.dimacs	TLE	TLE	36.138
TCTD.dimacs	TLE	TLE	123.680
TCTM.dimacs	TLE	TLE	618.628
TCTV.dimacs	TLE	TLE	$\sim 6930.000^\dagger$

TLE = Time Limit Exceeded ($> 3h$).

† Projected value based on empirical quadratic scaling ($O((m+n)^2)$).

5.2 Summary

The Edmonds-Karp capacity scaling algorithm yields the best running times in almost every tested benchmark, though the cost scaling algorithm performs comparably well in many cases.

Although Orlin's algorithm is theoretically faster, its practical implementation is generally much slower compared to other algorithms. The required network transformation appears to be its primary performance bottleneck.

Typically, the most time-consuming operation in Orlin's algorithm is finding augmenting paths. Despite optimizations to the augmenting arcs incident to uncapacitated vertices, the transformed networks remain too large to achieve reasonable running times.

However, if we introduced some uncapacitated instances, Orlin's algorithm would likely be faster compared to the standard capacity scaling algorithm by Edmonds and Karp, as the network transformation would not be necessary.

Capacity scaling algorithms may also be more efficient for dense networks with a single strongly connected component. The need to augment flow through an artificial node, created to ensure network connectivity, forces us to augment an equal amount of flow outward. Therefore, if a network contains multiple components, the nodes for augmentation must be chosen more carefully.

Bibliography

- [1] Srećko Krile. “Application of the minimum cost flow problem in container shipping”. In: *Proceedings ELMAR-2004: 46th International Symposium Electronics in Marine*. Ed. by Tomislav Kos and Mislav Grgić. Croatian Society Electronics in Marine, 2004, pp. 466–471. ISBN: 953-7044-02-5. DOI: 10.1109/ELMAR.2004.1356421.
- [2] Dirk Padfield, Jens Rittscher, and Badrinath Roysam. “Coupled minimum-cost flow cell tracking for high-throughput quantitative analysis”. In: *Medical Image Analysis* 15.4 (2011), pp. 650–668. DOI: 10.1016/j.media.2010.07.006.
- [3] Charles Werner, Urs Wegmüller, and Tazio Strozzi. “Processing strategies for phase unwrapping for InSAR applications”. In: *Proceedings of the 4th European Conference on Synthetic Aperture Radar (EUSAR 2002)*. Vol. 1. VDE Verlag, 2002, pp. 353–356. ISBN: 3-8007-2697-1.
- [4] Jennifer Fleming et al. “AlphaFold Protein Structure Database and 3D-Beacons: New data and capabilities”. In: *Journal of Molecular Biology* 437.15 (2025), p. 168967. DOI: 10.1016/j.jmb.2025.168967.
- [5] Radoslav Krivák and David Hoksza. “P2Rank: Machine learning based tool for rapid and accurate prediction of ligand binding sites from protein structure”. In: *Journal of Cheminformatics* 10.1 (2018), p. 39. DOI: 10.1186/s13321-018-0285-8.
- [6] Masao Iri. “A new method of solving transportation-network problems”. In: *Journal of the Operations Research Society of Japan* 3 (1960), pp. 27–87.
- [7] Robert Busacker and Paul Gowen. *A procedure for determining a family of minimum-cost network flow patterns*. Tech. rep. Technical Paper 15. Operations Research Office, Johns Hopkins University, 1960.
- [8] George Minty. “Monotone networks”. In: *Proceedings of the Royal Society of London. Series A. Mathematical and Physical Sciences* 257.1289 (1960), pp. 194–212. DOI: 10.1098/rspa.1960.0144.
- [9] Delbert Fulkerson. “An out-of-kilter method for minimal-cost flow problems”. In: *Journal of the Society for Industrial and Applied Mathematics* 9.1 (1961), pp. 18–27. DOI: 10.1137/0109002.

- [10] Jr. Ford Lester and Delbert Fulkerson. *Flows in Networks*. Princeton University Press, 1962.
- [11] Morton Klein. “A primal method for minimal cost flows with applications to the assignment and transportation problems”. In: *Management Science* 14.3 (1967), pp. 205–220. DOI: 10.1287/mnsc.14.3.205.
- [12] Jack Edmonds and Richard Karp. “Theoretical improvements in algorithmic efficiency for network flow problems”. In: *Combinatorial Structures and Their Applications*. Ed. by Richard Guy, Haim Hanani, Norbert Sauer, and J. Schönheim. Gordon and Breach, 1970, pp. 93–96.
- [13] Oliverio Cruz-Mejía and Adam Letchford. “A survey on exact algorithms for the maximum flow and minimum-cost flow problems”. In: *Networks* 82.2 (2023), pp. 167–176. DOI: 10.1002/net.22169.
- [14] Jack Edmonds and Richard Karp. “Theoretical improvements in algorithmic efficiency for network flow problems”. In: *Journal of the ACM* 19.2 (1972), pp. 248–264. DOI: 10.1145/321694.321699.
- [15] Uwe Pape, ed. *Discrete Structures and Algorithms: Proceedings of the Workshop WG ’79, 5th Conference on Graphtheoretic Concepts in Computer Science, Berlin (West), Germany, June 21–23, 1979*. Carl Hanser Verlag, 1980. ISBN: 3-446-13135-3.
- [16] James Orlin. *Genuinely polynomial simplex and non-simplex algorithms for the minimum cost flow problem*. Tech. rep. Sloan W.P. No. 1615-84. MIT Sloan School of Management, 1984.
- [17] Satoru Fujishige. “A capacity-rounding algorithm for the minimum-cost circulation problem: A dual framework of the Tardos algorithm”. In: *Mathematical Programming* 35.3 (1986), pp. 298–308. DOI: 10.1007/BF01580882.
- [18] Éva Tardos. “A strongly polynomial minimum cost circulation algorithm”. In: *Combinatorica* 5.3 (1985), pp. 247–255. DOI: 10.1007/BF02579369.
- [19] Zvi Galil and Éva Tardos. “An $O(n^2(m + n \log n) \log n)$ min-cost flow algorithm”. In: *Journal of the ACM* 35.2 (1988), pp. 374–386. DOI: 10.1145/42282.214090.
- [20] Harold Gabow and Robert Tarjan. “Faster scaling algorithms for network problems”. In: *SIAM Journal on Computing* 18.5 (1989), pp. 1013–1036. DOI: 10.1137/0218069.
- [21] Andrew Goldberg and Robert Tarjan. “Finding minimum-cost circulations by successive approximation”. In: *Mathematics of Operations Research* 15.3 (1990), pp. 430–466. DOI: 10.1287/moor.15.3.430.
- [22] James Orlin. “A faster strongly polynomial minimum cost flow algorithm”. In: *Operations Research* 41.2 (1993), pp. 338–350. DOI: 10.1287/opre.41.2.338.
- [23] Ravindra Ahuja, Andrew Goldberg, James Orlin, and Robert Tarjan. “Finding minimum-cost flows by double scaling”. In: *Mathematical Programming* 53.1–3 (1992), pp. 243–266. DOI: 10.1007/BF01585705.

- [24] Samuel Daitch and Daniel Spielman. “Faster approximate lossy generalized flow via interior point algorithms”. In: *Proceedings of the 40th Annual ACM Symposium on Theory of Computing (STOC 2008)*. Ed. by Richard Ladner and Cynthia Dwork. ACM, 2008, pp. 451–460. ISBN: 978-1-60558-047-0. DOI: 10.1145/1374376.1374441.
- [25] Yin Tat Lee and Aaron Sidford. “Path-finding methods for linear programming: Solving linear programs in $\tilde{O}(\sqrt{\text{rank}})$ iterations and faster algorithms for maximum flow”. In: *2014 IEEE 55th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE Computer Society, 2014, pp. 424–433. ISBN: 978-1-4799-6517-5. DOI: 10.1109/FOCS.2014.52.
- [26] Jan van den Brand et al. “Minimum cost flows, MDPs, and ℓ_1 -regression in nearly linear time for dense instances”. In: *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing (STOC 2021)*. Ed. by Samir Khuller and Virginia Vassilevska Williams. ACM, 2021, pp. 859–869. ISBN: 978-1-4503-8053-9. DOI: 10.1145/3406325.3451108.
- [27] Kyriakos Axiotis, Aleksander Mądry, and Adrian Vladu. “Faster sparse minimum cost flow by electrical flow localization”. In: *2021 IEEE 62nd Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 2021, pp. 528–539. DOI: 10.1109/FOCS52979.2021.00059.
- [28] Jan van den Brand et al. “Faster maxflow via improved dynamic spectral vertex sparsifiers”. In: *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing (STOC 2022)*. Ed. by Stefano Leonardi and Anupam Gupta. ACM, 2022, pp. 543–556. ISBN: 978-1-4503-9264-8. DOI: 10.1145/3519935.3520068.
- [29] Li Chen et al. “Maximum flow and minimum-cost flow in almost-linear time”. In: *Journal of the ACM* 72.3 (2025), 19:1–19:103. DOI: 10.1145/3728631.
- [30] Ravindra Ahuja, Thomas Magnanti, and James Orlin. *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993. ISBN: 9780136175490.
- [31] David Williamson. *Network Flow Algorithms*. Cambridge University Press, 2019. ISBN: 9781316636831.
- [32] Richard Bellman. “On a routing problem”. In: *Quarterly of Applied Mathematics* 16.1 (1958), pp. 87–90. DOI: 10.1090/qam/102435.
- [33] Michael Fredman and Robert Tarjan. “Fibonacci heaps and their uses in improved network optimization algorithms”. In: *Journal of the ACM* 34.3 (1987), pp. 596–615. DOI: 10.1145/28869.28874.
- [34] Darwin Klingman, Albert Napier, and Joel Stutz. “NETGEN: A program for generating large scale capacitated assignment, transportation, and minimum cost flow network problems”. In: *Management Science* 20.5 (1974), pp. 814–821. DOI: 10.1287/mnsc.20.5.814.
- [35] Yusin Lee and James Orlin. *GRIDGEN*. 1991. URL: <https://archive.dimacs.rutgers.edu/archive/pub/netflow/generators/network/gridgen/>.

- [36] Stephan Held. *Large Benchmarks for the Minimum-Cost Flow Problem*. Version V1. bonndata, 2024. DOI: 10.60507/FK2/5IKK7K.